



SMART CONTRACT AUDIT REPORT

for

StableSwap Hooks



Prepared By: Xiaomi Huang

PeckShield
January 29, 2026

Document Properties

Client	Revert
Title	Smart Contract Audit Report
Target	StableSwap Hooks
Version	1.0
Author	Xuxian Jiang
Auditors	Matthew Jiang, Xuxian Jiang
Reviewed by	Xiaomi Huang
Approved by	Xuxian Jiang
Classification	Public

Version Info

Version	Date	Author(s)	Description
1.0	January 29, 2026	Xuxian Jiang	Final Release
1.0-rc	January 19, 2026	Xuxian Jiang	Release Candidate #1

Contact

For more information about this document and its contents, please contact PeckShield Inc.

Name	Xiaomi Huang
Phone	+86 183 5897 7782
Email	contact@peckshield.com

Contents

1	Introduction	4
1.1	About StableSwap Hooks	4
1.2	About PeckShield	5
1.3	Methodology	5
1.4	Disclaimer	7
2	Findings	9
2.1	Summary	9
2.2	Key Findings	10
3	Detailed Results	11
3.1	Revisited Hook Permissions in Base	11
3.2	Improved Precision in Liquidity Addition Calculation	12
3.3	Lack of Native ETH Support in StableSwapHooks	14
3.4	Suggested Adherence Of Checks-Effects-Interactions Pattern	16
3.5	Missing Initialization of First Amp Start/Stop Time	17
4	Conclusion	19
	References	20

1 | Introduction

Given the opportunity to review the design document and related source code of the `StableSwap Hooks` protocol, we outline in the report our systematic approach to evaluate potential security issues in the smart contract implementation, expose possible semantic inconsistencies between smart contract code and design document, and provide additional suggestions or recommendations for improvement. Our results show that the given version of smart contracts can be further improved due to the presence of several issues related to either security or performance. This document outlines our audit results.

1.1 About StableSwap Hooks

`StableSwap Hooks` is a `UniswapV4` hook implementation that brings `Curve`-style `StableSwap` AMM behavior to `UniswapV4` pools. It targets stable assets (stablecoins, LSTs, etc.) to provide low-slippage swaps around the 1:1 price, while supporting configurable fees, rate oracles, and amplification (A) ramping. The basic information of the audited protocol is as follows:

Table 1.1: Basic Information of StableSwap Hooks

Item	Description
Name	Revert
Type	Smart Contract
Language	Solidity
Audit Method	Whitebox
Latest Audit Report	January 29, 2026

In the following, we show the Git repository of reviewed files and the commit hash value used in this audit.

- <https://github.com/BootNodeDev/stableswap-hooks.git> (f22a7f2)

And here is the commit hash value after all fixes for the issues found in the audit have been applied:

- <https://github.com/BootNodeDev/stableswap-hooks.git> (4ba66b7)

1.2 About PeckShield

PeckShield Inc. [11] is a leading blockchain security company with the goal of elevating the security, privacy, and usability of current blockchain ecosystems by offering top-notch, industry-leading services and products (including the service of smart contract auditing). We are reachable at Telegram (<https://t.me/peckshield>), Twitter (<http://twitter.com/peckshield>), or Email (contact@peckshield.com).

Table 1.2: Vulnerability Severity Classification

Impact	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low
		High	Medium	Low
		Likelihood		

1.3 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [10]:

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;
- Impact measures the technical loss and business damage of a successful attack;
- Severity demonstrates the overall criticality of the risk.

Likelihood and impact are categorized into three ratings: *H*, *M* and *L*, i.e., *high*, *medium* and *low* respectively. Severity is determined by likelihood and impact, and can be accordingly classified into four categories, i.e., *Critical*, *High*, *Medium*, *Low* shown in Table 1.2.

To evaluate the risk, we go through a list of check items and each would be labeled with a severity category. For one check item, if our tool or analysis does not identify any issue, the contract is considered safe regarding the check item. For any discovered issue, we might further

Table 1.3: The Full List of Check Items

Category	Check Item
Basic Coding Bugs	Constructor Mismatch
	Ownership Takeover
	Redundant Fallback Function
	Overflows & Underflows
	Reentrancy
	Money-Giving Bug
	Blackhole
	Unauthorized Self-Destruct
	Revert DoS
	Unchecked External Call
	Gasless Send
	Send Instead Of Transfer
	Costly Loop
	(Unsafe) Use Of Untrusted Libraries
	(Unsafe) Use Of Predictable Variables
	Transaction Ordering Dependence
	Deprecated Uses
Semantic Consistency Checks	Semantic Consistency Checks
Advanced DeFi Scrutiny	Business Logics Review
	Functionality Checks
	Authentication Management
	Access Control & Authorization
	Oracle Security
	Digital Asset Escrow
	Kill-Switch Mechanism
	Operation Trails & Event Generation
	ERC20 Idiosyncrasies Handling
	Frontend-Contract Integration
	Deployment Consistency
	Holistic Risk Management
Additional Recommendations	Avoiding Use of Variadic Byte Array
	Using Fixed Compiler Version
	Making Visibility Level Explicit
	Making Type Inference Explicit
	Adhering To Function Declaration Strictly
	Following Other Best Practices

deploy contracts on our private testnet and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.3.

In particular, we perform the audit according to the following procedure:

- Basic Coding Bugs: We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- Semantic Consistency Checks: We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
- Advanced DeFi Scrutiny: We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- Additional Recommendations: We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [9], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development. Though some categories used in CWE-699 may not be relevant in smart contracts, we use the CWE categories in Table 1.4 to classify our findings. Moreover, in case there is an issue that may affect an active protocol that has been deployed, the public version of this report may omit such issue, but will be amended with full details right after the affected protocol is upgraded with respective fixes.

1.4 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, we always recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contract(s). Last but not least, this security audit should not be used as investment advice.

Table 1.4: Common Weakness Enumeration (CWE) Classifications Used in This Audit

Category	Summary
Configuration	Weaknesses in this category are typically introduced during the configuration of the software.
Data Processing Issues	Weaknesses in this category are typically found in functionality that processes data.
Numeric Errors	Weaknesses in this category are related to improper calculation or conversion of numbers.
Security Features	Weaknesses in this category are concerned with topics like authentication, access control, confidentiality, cryptography, and privilege management. (Software security is not security software.)
Time and State	Weaknesses in this category are related to the improper management of time and state in an environment that supports simultaneous or near-simultaneous computation by multiple systems, processes, or threads.
Error Conditions, Return Values, Status Codes	Weaknesses in this category include weaknesses that occur if a function does not generate the correct return/status code, or if the application does not handle all possible return/status codes that could be generated by a function.
Resource Management	Weaknesses in this category are related to improper management of system resources.
Behavioral Issues	Weaknesses in this category are related to unexpected behaviors from code that an application uses.
Business Logics	Weaknesses in this category identify some of the underlying problems that commonly allow attackers to manipulate the business logic of an application. Errors in business logic can be devastating to an entire application.
Initialization and Cleanup	Weaknesses in this category occur in behaviors that are used for initialization and breakdown.
Arguments and Parameters	Weaknesses in this category are related to improper use of arguments or parameters within function calls.
Expression Issues	Weaknesses in this category are related to incorrectly written expressions within code.
Coding Practices	Weaknesses in this category are related to coding practices that are deemed unsafe and increase the chances that an exploitable vulnerability will be present in the application. They may not directly introduce a vulnerability, but indicate the product has not been carefully developed or maintained.

2 | Findings

2.1 Summary

Here is a summary of our findings after analyzing the implementation of the `StableSwap Hooks` protocol. During the first phase of our audit, we study the smart contract source code and run our in-house static code analyzer through the codebase. The purpose here is to statically identify known coding bugs, and then manually verify (reject or confirm) issues reported by our tool. We further manually review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.

Severity	# of Findings	
Critical	0	
High	0	
Medium	0	
Low	4	
Informational	1	
Total	5	

We have so far identified a list of potential issues: some of them involve subtle corner cases that might not be previously thought of. For each uncovered issue, we have therefore developed test cases for reasoning, reproduction, and/or verification. After further analysis and internal discussion, we determined a few issues of varying severities need to be brought up and paid more attention to, which are categorized in the above table. More information can be found in the next subsection, and the detailed discussions of each of them are in [Section 3](#).

2.2 Key Findings

Overall, this smart contract is well-designed and engineered, though the implementation can be improved by resolving the identified issues (shown in Table 2.1), including 4 low-severity vulnerabilities and 1 informational suggestion.

Table 2.1: Key StableSwap Hooks Audit Findings

ID	Severity	Title	Category	Status
PVE-001	Informational	Revisited Hook Permissions in Base	Business Logic	Resolved
PVE-002	Low	Improved Precision in Liquidity Addition Calculation	Numeric Issues	Resolved
PVE-003	Low	Lack of Native ETH Support in StableSwapHooks	Business Logic	Resolved
PVE-004	Low	Suggested Adherence of Checks-Effects-Interactions	Time and State	Resolved
PVE-005	Low	Missing Initialization of First Amp Start/Stop Time	Init. And Cleanup	Resolved

Beside the identified issues, we emphasize that for any user-facing applications and services, it is always important to develop necessary risk-control mechanisms and make contingency plans, which may need to be exercised before the mainnet deployment. The risk-control mechanisms should kick in at the very moment when the contract is being deployed on mainnet. Please refer to Section 3 for details.

3 | Detailed Results

3.1 Revisited Hook Permissions in Base

- ID: PVE-001
- Severity: Informational
- Likelihood: N/A
- Impact: N/A
- Target: Base
- Category: Business Logic [6]
- CWE subcategory: CWE-770 [4]

Description

As mentioned earlier, `StableSwap Hooks` are modular smart contract “plugins” that are used to customize and extend the behavior of `UniswapV4` liquidity pools without modifying the core protocol logic. With the purpose to utilize the `Curve-style StableSwap AMM` swap behavior, it declares the following six hook permissions, i.e., `beforeInitialize`, `beforeAddLiquidity`, `beforeRemoveLiquidity`, `beforeSwap`, `beforeSwapReturnDelta`, and `beforeDonate`. Our analysis shows the specific `beforeRemoveLiquidity` hook is not necessary in our scenario.

To elaborate, we show below the implementation of the related `getHookPermissions()` function. Note the `beforeRemoveLiquidity` hook runs before any liquidity is withdrawn from the pool. By design, all liquidity is separately managed via the `Liquidity` contract, not through `UniswapV4` logic. And the `beforeAddLiquidity` hook is used to simply block the liquidity addition via the `UniswapV4` front. With that, the `beforeRemoveLiquidity` hook becomes not necessary (as there is no liquidity to be added in the first place via `UniswapV4`) and has the benefit of relaxing the hook address discovery.

From another perspective, both `beforeAddLiquidity` and `beforeRemoveLiquidity` hooks typically show up together when there is a need to restrict or modify how liquidity is handled. The former decides who can enter and update the initial liquidity state while the latter enforces exit rules and cleans up the resulting liquidity state.

```
158     function getHookPermissions() public pure override returns (Hooks.Permissions memory
      permissions) {
159         permissions = Hooks.Permissions({
```

```

160         beforeInitialize: true,
161         afterInitialize: false,
162         beforeAddLiquidity: true,
163         afterAddLiquidity: false,
164         beforeRemoveLiquidity: true,
165         afterRemoveLiquidity: false,
166         beforeSwap: true,
167         afterSwap: false,
168         beforeDonate: true,
169         afterDonate: false,
170         beforeSwapReturnDelta: true,
171         afterSwapReturnDelta: false,
172         afterAddLiquidityReturnDelta: false,
173         afterRemoveLiquidityReturnDelta: false
174     });
175 }

```

Listing 3.1: Base::getHookPermissions()

Recommendation Revisit the above hook permission flags and remove the redundant `beforeRemoveLiquidity` hook.

Status This issue has been resolved as it is part of the design. The purpose here is to explicitly signal that the hook is unsupported, even though any attempt to use it would revert.

3.2 Improved Precision in Liquidity Addition Calculation

- ID: PVE-002
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: Liquidity
- Category: Numeric Issue [8]
- CWE subcategory: CWE-190 [2]

Description

`SafeMath` is a widely-used Solidity `math` library that is designed to support safe `math` operations by preventing common overflow or underflow issues when working with `uint256` operands. While it indeed blocks common overflow or underflow issues, the lack of `float` support in `Solidity` may introduce another subtle, but troublesome issue: precision loss. In this section, we examine one possible precision loss source that stems from the flooring/ceiling choices when both multiplication (`mul`) and division (`div`) are involved.

In particular, we use the `Liquidity::_calculateAddLiquidity()` as an example. This routine is used to calculate shares and actual amounts for adding liquidity.

```

188     function _calculateAddLiquidity(uint256[] memory _amounts)
189     private
190     view
191     returns (uint256 shares, uint256[] memory actualAmounts)
192     {
193         uint256 currentTotalSupply = totalSupply();
194         actualAmounts = new uint256[](currenciesLength);

196         if (currentTotalSupply == 0) {
197             uint256[] memory scaledAmounts = new uint256[](currenciesLength);

199             for (uint256 i = 0; i < currenciesLength; ++i) {
200                 scaledAmounts[i] = StableSwapMath.scaleTo(_amounts[i], _getRate(i));
201             }

203             shares = StableSwapMath.geometricMean(scaledAmounts);

205             if (shares < MINIMUM_LIQUIDITY) {
206                 revert InsufficientInitialLiquidity();
207             }

209             shares -= MINIMUM_LIQUIDITY;

211             for (uint256 i = 0; i < currenciesLength; ++i) {
212                 actualAmounts[i] = _amounts[i];
213             }
214         } else {
215             uint256[] memory cachedRates = new uint256[](currenciesLength);
216             uint256[] memory scaledReserves = new uint256[](currenciesLength);

218             for (uint256 i = 0; i < currenciesLength; ++i) {
219                 cachedRates[i] = _getRate(i);
220                 scaledReserves[i] = StableSwapMath.scaleTo(reserves[i], cachedRates[i]);
221             }

223             uint256 minProportion = type(uint256).max;

225             for (uint256 i = 0; i < currenciesLength; ++i) {
226                 uint256 scaledAmount = StableSwapMath.scaleTo(_amounts[i], cachedRates[i]);
227                 uint256 proportion = Math.mulDiv(scaledAmount, currentTotalSupply, scaledReserves[i]);

229                 if (proportion < minProportion) {
230                     minProportion = proportion;
231                 }
232             }

234             shares = minProportion;

236             for (uint256 i = 0; i < currenciesLength; ++i) {
237                 uint256 scaledAmount = (minProportion * scaledReserves[i]) /

```

```

238         currentTotalSupply;
239         actualAmounts[i] = StableSwapMath.descale(scaledAmount, cachedRates[i]);
240     }
241 }

```

Listing 3.2: Liquidity :: _calculateAddLiquidity()

We notice the calculation of the actual amounts for adding liquidity (lines 237 – 238) involves a `descale()` call to compute the amounts in token decimals. The calculation is performed by taking a flooring of the actual amount, which is suggested to take a ceiling in favor of the protocol. In the meantime, the use of `scaledAmount` can be avoided with the direct use of `reserves`. By doing so, we avoid introducing possible precision loss with the use of an intermediate variable.

Recommendation Revise the above calculations to better mitigate possible precision loss, especially with the choice of in favor of the protocol.

Status The issue has been fixed by this commit: 4ba66b7.

3.3 Lack of Native ETH Support in StableSwapHooks

- ID: PVE-003
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: Liquidity
- Category: Business Logic [6]
- CWE subcategory: CWE-770 [4]

Description

The StableSwap Hooks protocol is designed to bring Curve-style StableSwap AMM behavior to UniswapV4 pools. And it targets stable assets (stablecoins, LSTs, etc.) to provide low-slippage swaps around the 1:1 price. Our analysis on possible LSTs support shows current implementation can be improved by supporting native ETH.

In the following, we show the execution logic of adding liquidity. It basically unlocks UniswapV4's `poolManager`, which makes a callback and hands over the execution back to the hook contract, which eventually executes the following `_handleAddLiquidityCallback()` function. After internally computing the required amounts for underlying tokens, it iterates the list of underlying tokens and makes the following call sequence, i.e., `sync-transferFrom-settle-mint` (lines 144-151). Apparently, the use of `transferFrom()` is only applicable to regular ERC20 tokens.

```

74     function addLiquidity(uint256[] calldata _amounts, uint256[] calldata _minAmounts,
75         uint256 _minShares) external {
    bytes memory data = abi.encode(Actions.ADD_LIQUIDITY, _amounts, _minAmounts,
        _minShares, msg.sender);

```

```

77     poolManager.unlock(data);
78 }

```

Listing 3.3: Liquidity::addLiquidity()

```

122 function _handleAddLiquidityCallback(bytes calldata data) internal {
123     (, uint256[] memory amounts, uint256[] memory minAmounts, uint256 minShares,
124         address sender) =
125         abi.decode(data, (uint256, uint256[], uint256[], uint256, address));
126
127     bool isInitialDeposit = totalSupply() == 0;
128
129     (uint256 newShares, uint256[] memory actualAmounts) = _calculateAddLiquidity(
130         amounts);
131
132     if (newShares < minShares) {
133         revert InsufficientShares();
134     }
135
136     for (uint256 i = 0; i < currenciesLength; ++i) {
137         if (actualAmounts[i] < minAmounts[i]) {
138             revert InsufficientAmountsUsed();
139         }
140     }
141
142     if (isInitialDeposit) {
143         _mint(DEAD_ADDRESS, MINIMUM_LIQUIDITY);
144     }
145
146     for (uint256 i = 0; i < currenciesLength; ++i) {
147         Currency currency = currencies[i];
148         poolManager.sync(currency);
149         IERC20(Currency.unwrap(currency)).safeTransferFrom(sender, address(
150             poolManager), actualAmounts[i]);
151         poolManager.settle();
152         poolManager.mint(address(this), currency.toId(), actualAmounts[i]);
153         reserves[i] += actualAmounts[i];
154     }
155
156     _mint(sender, newShares);
157
158     emit LiquidityAdded(sender, actualAmounts, newShares);
159 }

```

Listing 3.4: Liquidity::_handleAddLiquidityCallback()

To support native ETH, we can piggy back the `msg.value` in the callback data, which can be examined against the `ADDRESS_ZERO` currency, and then directly transfers the native ETH (in the amount of `actualAmounts[i]`) along with the `settle()` call. After that, we may need to refund extra ETHs, if any, back to the user. And the refund amount can be calculated by deducting the `actualAmounts[i]`

from the piggy-backed `msg.value` amount.

Recommendation Add the native ETH support to better accommodate various liquidity assets.

Status The issue has been fixed by this commit: 85000f5.

3.4 Suggested Adherence Of Checks-Effects-Interactions Pattern

- ID: PVE-004
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: Fees
- Category: Time and State [7]
- CWE subcategory: CWE-663 [3]

Description

A common coding best practice in Solidity is the adherence of checks-effects-interactions principle. This principle is effective in mitigating a serious attack vector known as re-entrancy. Via this particular attack vector, a malicious contract can be reentering a vulnerable contract in a nested manner. Specifically, it first calls a function in the vulnerable contract, but before the first instance of the function call is finished, second call can be arranged to re-enter the vulnerable contract by invoking functions that should only be executed once. This attack was part of several most prominent hacks in Ethereum history, including the DAO [13] exploit, and the Uniswap/Lendf.Me hack [12].

We notice there are occasions where the checks-effects-interactions principle is violated. Using the Fees as an example, the `_handleWithdrawProtocolFeesCallback()` function (see the code snippet below) is provided to externally call a contract to transfer assets. However, the invocation of an external contract requires extra care in avoiding the above re-entrancy. For example, the interaction with the external contract (line 104) start before effecting the update on internal states, hence violating the principle. In this particular case, if the external contract has certain hidden logic that may be capable of launching re-entrancy via the same entry function. Fortunately, the caller of this specific function is strict in disallowing the re-entrancy.

```

98     function _handleWithdrawProtocolFeesCallback(bytes calldata data) internal {
99         (, address sender) = abi.decode(data, (uint256, address));
100
101         address _protocolFeeCollector = factory.protocolFeeCollector();
102         uint256[] memory _protocolFees = protocolFees;
103
104         _handleWithdrawFeesPoolManagerAccounting(_protocolFeeCollector, _protocolFees);
105
106         for (uint256 i = 0; i < currenciesLength; i++) {

```

```

107     protocolFees[i] = 0;
108 }
109
110     emit ProtocolFeesWithdrawn(sender, _protocolFeeCollector, _protocolFees);
111 }

```

Listing 3.5: Fees::_handleWithdrawProtocolFeesCallback()

While current implementation is not vulnerable or exploitable for re-entrancy, it is still important to take precautions to thwart possible re-entrancy risks. With that, it is strongly suggested to follow the best practices of the checks-effects-interactions principle.

Recommendation Revisit the above function to follow the checks-effects-interactions principle. Note another function `_handleWithdrawHookFeesCallback()` can be similarly improved.

Status The issue has been fixed by this commit: 2b38ba2.

3.5 Missing Initialization of First Amp Start/Stop Time

- ID: PVE-005
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: Amp
- Category: Coding Practices [5]
- CWE subcategory: CWE-1126 [1]

Description

The StableSwap AMM logic has a built-in amplification coefficient (A), which controls how tightly the pool prices around the 1:1 peg. Higher A reduces slippage near equilibrium, while lower A behaves closer to constant product. The factory owner can update A over time using ramping. The current A value is calculated by interpolating `baseAmp` and `nextAmp` over `baseAmpTime` and `nextAmpTime`. Our analysis shows the very first `baseAmpTime` and `nextAmpTime` upon the contract deployment are not initialized.

To elaborate, we show below the constructor of the related `Amp` contract. While it properly configures `baseAmp` and `nextAmp`, it does not configure `baseAmpTime` and `nextAmpTime`. While it allows for immediately starting a new gradual ramp of the amplification coefficient to a new value, it logically makes better sense if we consider the constructor function just finishes the last-round ramp of `nextAmp` at `nextAmpTime`. With that, we suggest to initialize these two timestamps upon deployment to `block.timestamp`.

```

53     constructor(uint256 _baseAmp) {
54         if (_baseAmp >= MAX_AMP) {
55             revert InvalidAmp();

```

```
56     }  
57  
58     uint256 scaledBaseAmp = _baseAmp * StableSwapMath.AMP_PRECISION;  
59  
60     baseAmp = scaledBaseAmp;  
61     nextAmp = scaledBaseAmp;  
62 }
```

Listing 3.6: `Amp::constructor()`

Recommendation Revisit the above constructor to initialize both `baseAmpTime` and `nextAmpTime` to `block.timestamp`.

Status This issue has been resolved as it is consistent with the original `Curve` implementation.



4 | Conclusion

In this audit, we have analyzed the design and implementation of the `StableSwap Hooks` protocol, a `UniswapV4` hook implementation that brings `Curve`-style `StableSwap` AMM behavior to `UniswapV4` pools. It targets stable assets (stablecoins, LSTs, etc.) to provide low-slippage swaps around the 1:1 price, while supporting configurable fees, rate oracles, and amplification (A) ramping. The current code base is well structured and neatly organized. Those identified issues are promptly confirmed and addressed.

Meanwhile, we need to emphasize that `Solidity`-based smart contracts as a whole are still in an early, but exciting stage of development. To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.



References

- [1] MITRE. CWE-1126: Declaration of Variable with Unnecessarily Wide Scope. <https://cwe.mitre.org/data/definitions/1126.html>.
- [2] MITRE. CWE-190: Integer Overflow or Wraparound. <https://cwe.mitre.org/data/definitions/190.html>.
- [3] MITRE. CWE-663: Use of a Non-reentrant Function in a Concurrent Context. <https://cwe.mitre.org/data/definitions/663.html>.
- [4] MITRE. CWE-770: Allocation of Resources Without Limits or Throttling. <https://cwe.mitre.org/data/definitions/770.html>.
- [5] MITRE. CWE CATEGORY: Bad Coding Practices. <https://cwe.mitre.org/data/definitions/1006.html>.
- [6] MITRE. CWE CATEGORY: Business Logic Errors. <https://cwe.mitre.org/data/definitions/840.html>.
- [7] MITRE. CWE CATEGORY: Concurrency. <https://cwe.mitre.org/data/definitions/557.html>.
- [8] MITRE. CWE CATEGORY: Numeric Errors. <https://cwe.mitre.org/data/definitions/189.html>.
- [9] MITRE. CWE VIEW: Development Concepts. <https://cwe.mitre.org/data/definitions/699.html>.

- [10] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology.
- [11] PeckShield. PeckShield Inc. <https://www.peckshield.com>.
- [12] PeckShield. Uniswap/Lendf.Me Hacks: Root Cause and Loss Analysis. <https://medium.com/@peckshield/uniswap-lendf-me-hacks-root-cause-and-loss-analysis-50f3263dcc09>.
- [13] David Siegel. Understanding The DAO Attack. <https://www.coindesk.com/understanding-dao-hack-journalists>.

